ZARIERA

EVERY JEWEL TELLS A STORY

Project Handbook & Technical Documentation

Premium stainless-steel & artificial jewellery e-commerce

Built with Next.js 16 · TypeScript · Prisma · PostgreSQL

Document date: 29 September 2026 · Version 1.0

1. Table of contents

1. Project overview
2. Technology stack
3. Running the project (setup & scripts)
4. Architecture & directory map
5. Database schema (models & enums)
6. Environment variables
7. Storefront features (customer side)
8. Admin panel & dashboard features
9. Routes & API endpoints
10. Component & library inventory
11. Design system & conventions
12. Known gotchas & fixes
13. Current data state
14. Deployment checklist
15. Pending / next tasks

2. Project overview

ZARIERA is a full e-commerce store for **stainless-steel / artificial (fashion) jewellery** — necklaces, rings, earrings, bracelets and bangles. It is *not* a real-gold store. Currency is **PKR (Rs)**. It is being built for a Pakistani client (Saad) and is intended to launch in Pakistan with **Cash-on-Delivery + online payments** and **dropshipping** fulfilment via **Leopards** and **TCS** couriers.

It has two halves: a polished, fully-responsive **storefront** (browse → cart → checkout → account), and a complete **admin panel** (products, orders, customers, categories, shipping, and a rich analytics dashboard).

> **Positioning:** premium / "international" visual quality — deep maroon & gold palette, serif display type, dark-mode support, smooth interactions, and analytics that feel like Shopify / Amazon Seller Central.

3. Technology stack

Layer	Technology
Framework	Next.js 16.3.5 — App Router, route groups, server actions, `useActionState`, Turbopack
Language / UI	TypeScript · React 19.2
Styling	Tailwind CSS v4 (CSS `@theme` tokens, `@custom-variant dark`)
Database / ORM	PostgreSQL + Prisma 6.19.3
Charts	lightweight-charts 4.2.3 (TradingView) for the zoomable sales graph; hand-built SVG for the donut & radial gauges
Email	nodemailer 10 (best-effort order emails)
Validation	zod 4
Auth	Custom — HMAC-signed session cookies, scrypt password hashing (no external auth dependency)

4. Running the project

```
cd C:/Users/Gigalabs/Downloads/zariera
npm install
npm run dev        # http://localhost:3000  (Turbopack dev server)
```

URLs: Storefront `http://localhost:3000` · Admin `http://localhost:3000/admin` (login at `/admin/login`).

Script	Purpose
`npm run dev`	Start the dev server (Turbopack)
`npm run build` / `start`	Production build / serve
`npm run lint`	ESLint
`npm run db:migrate` / `db:push`	Prisma migrate / push schema
`npm run db:studio`	Prisma Studio (browse DB)
`npm run db:seed`	Seed sample data
`npm run db:create-admin`	Create the admin user from `ADMIN_EMAIL` / `ADMIN_PASSWORD` in `.env`

Always run `npx tsc --noEmit` after edits — it should exit 0.

5. Architecture & directory map

```
src/
  app/
    (store)/            storefront route group (home, shop, product, cart, checkout,
                        account, login, register, about, contact, faq, care, shipping, custom, search)
    admin/
      login/            admin authentication (separate from storefront)
      (dashboard)/      admin app, wrapped by AdminShell (collapsible sidebar + topbar)
        page.tsx        DASHBOARD (KPIs + all analytics widgets)
        analytics/      Sales graph + category mix
        top-products/   Best sellers (top 20)
        store-health/   Radial gauges + order-pipeline donut
        orders/ , orders/[id]/ , products/ , products/new/ , products/[id]/
        categories/ , customers/
    api/                orders, admin/orders/stream (SSE), admin/upload, payments/*
    layout.tsx          root: ToastProvider + FlashToast + CartProvider + theme no-flash script
  components/           admin/*, cart/*, ui/toast + flash-toast, and storefront components
  lib/                  data + server logic (Prisma queries, orders, auth, couriers, payments, email...)
prisma/schema.prisma    database schema
public/images/          product & marketing images (public/images/promo = 16 editorial shots)
```

6. Database schema

PostgreSQL via Prisma. Money fields are `Decimal(10,2)`.

Models

Model	Key fields
Category	id, name, slug, description, products[]

Product	id, name, slug, description, price, compareAtPrice, sku, stock, material, **specs** ("Label: Value" per line), **images[]**, featured, active, categoryId
Order	id, orderNumber (unique), status, paymentMethod, paymentStatus, customerName/email/phone/address/city/postalCode/notes, customerId?, subtotal, shipping, total, **courier**, **trackingNumber**, **shippedAt**, items[]
OrderItem	id, orderId, productId, name/price/quantity (snapshots). Cascade-deletes with its Order.
User	Admin account — id, email, passwordHash (scrypt), role
Customer	Storefront account — id, name, email, passwordHash, orders[]

Enums

- **OrderStatus**: PENDING · CONFIRMED · PROCESSING · SHIPPED · DELIVERED · CANCELLED
- **PaymentMethod**: COD · PAYFAST · SAFEPAY
- **PaymentStatus**: PENDING · PAID (+ related)
- **Role**: ADMIN

7. Environment variables

Stored in `.env` (gitignored). Courier & payment integrations fall back to **simulation/preview mode** when keys are absent, so the app runs without them during development.

Group	Variables
Auth / app	`AUTH_SECRET`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`, `NEXT_PUBLIC_SITE_URL`, `NODE_ENV`
Email	`SMTP_HOST/PORT/USER/PASS/SECURE`, `MAIL_FROM`, `ADMIN_NOTIFY_EMAIL`
Couriers	`LEOPARDS_API_KEY/API_PASSWORD/ORIGIN_CITY`, `TCS_USERNAME/PASSWORD/CLIENT_ID/CLIENT_SECRET/COST_CENTER/ORIGIN_CITY`
Payments	`PAYFAST_MERCHANT_ID/SECURED_KEY/SANDBOX`, `SAFEPAY_API_KEY/SANDBOX`

8. Storefront features (customer side)

- **Order flow (fixed):** `lib/orders.createOrder` re-prices every line against the DB (anti-tamper), validates stock, and creates the order + decrements stock **atomically** in a `$transaction`. Emails are best-effort. (An earlier bug that faked success on failure was removed.)
- **Guest checkout** — no account needed; optional account creation on the first order.
- **Premium header + marquee** — logo left, nav centred, icons right; hamburger on mobile.
- **Dark / light mode** — token flip via `data-theme`, no-flash inline script, remembered in `localStorage: zariera-theme`.
- **Fully responsive** on mobile & tablet across every page.
- **Home sections** — hero slider; "Signature Edit" editorial split slider (16 model/jewellery shots); shop-by-category image cards; trust badges; testimonials; Instagram; newsletter; premium footer.
- **Product detail (Amazon-style)** — two CTAs (Add to cart + Buy now), specifications table (from `product.specs`), collapsible accordions, hover-zoom gallery, sticky info rail.
- **Cart & checkout** — premium layout with a sticky/locked order summary.
- **Favicon** — the ZARIERA gold-lotus emblem (SVG + .ico + apple-icon).
- **Global toast notifications** — a reusable `useToast()` system wired into add-to-cart, buy-now, quick-add, cart remove, newsletter, checkout errors, login/register (success via post-redirect "flash" toasts), and logout.

9. Admin panel & dashboard

Shell & navigation

- **AdminShell** — collapsible sidebar (state saved in `localStorage: zariera-admin-collapsed`), sticky topbar with a sidebar toggle + live notification bell, and **independent scroll** (sidebar fixed, content scrolls; the frame uses `height:100dvh`).
- **Sidebar menu:** Dashboard · Analytics · Top products · Store health · Orders · Products · Categories · Customers.

Real-time order notifications

- Server-Sent Events stream at `/api/admin/orders/stream` fed by an in-memory bus (`lib/order-events.ts`) that `createOrder` emits to.
- Bell with unread badge, dropdown, toast, a "ding" (Web Audio), OS notification, and auto-reconnect.

Management

- **Orders** — premium table (status tabs + search); detail page with status/payment updates (with toasts) and **shipment booking** (Leopards/TCS; simulated tracking number until live keys are added).
- **Products** — full CRUD, drag-and-drop **image upload** (`/api/admin/upload` → `public/images/products/`), and a per-product **specifications** field.
- **Categories** & **Customers** — dedicated management pages.

Dashboard & analytics

- **8 KPI cards** — Revenue, This month, Orders, Month orders, Pending, Products, Customers, Low stock.
- **Sales performance** — Amazon/Keepa-style **zoomable graph** (lightweight-charts): mouse-wheel zoom, drag-pan, crosshair tooltip, range presets (1W/1M/3M/6M/1Y/All), zoom ± / reset buttons, and insight tiles that recompute for the visible window.
- **Store health** — three animated **radial gauges** (fulfilment %, payments collected %, catalog in stock %).
- **Order pipeline** — a Shopify-style interactive **donut**: hover any segment or legend row and it pops out (others dim), the centre shows that status's label + count + %, and a tooltip follows the cursor.
- **Top products** (best sellers), **Sales by category** (share bars), **New customers**, plus **Recent orders**, **Quick actions**, and a **Low-stock alert**.
- **Dedicated menu pages** — `/admin/analytics`, `/admin/top-products`, `/admin/store-health` each open a focused view reusing these widgets.

10. Routes & API endpoints

Storefront pages

- / (home)
- /shop, /shop/[category]
- /product/[slug]
- /cart, /checkout, /checkout/success
- /account, /account/orders
- /login, /register
- /search
- /about, /contact, /faq, /care, /shipping, /custom

Admin pages

- /admin (dashboard)
- /admin/analytics
- /admin/top-products
- /admin/store-health
- /admin/orders, /admin/orders/[id]
- /admin/products, /new, /[id]
- /admin/categories, /admin/customers
- /admin/login

API endpoints

Route	Purpose

`POST /api/orders`	Place an order (guest or logged-in) → `createOrder`
`GET /api/admin/orders/stream`	SSE stream of new-order events (admin only)
`POST /api/admin/upload`	Product image upload (admin only)
`/api/payments/payfast/callback`	PayFast payment callback
`/api/payments/safepay/webhook`	Safepay webhook

11. Component & library inventory

Admin components (`src/components/admin/`)

admin-shell · admin-nav · admin-mobile-nav · admin-login-form · dashboard-insights · order-pipeline-donut · sales-chart-pro · performance-rings · order-notifications · orders-table · status-badge · status-update-form · shipment-panel · product-form · category-form · db-banner · delete-button

Data layer (`src/lib/`)

prisma · admin-data · orders · order-events · couriers · payments · email · auth · admin · customer-auth · password · catalog · shipping · order-status · format · slug · types · sample-data

`admin-data.ts` holds every admin query/aggregation: stats, orders, products, customers, categories, sales daily series, sales insights (day/month/year), sales breakdown (top products + categories), order-status breakdown, and store performance.

12. Design system & conventions

Brand tokens live in `src/app/globals.css` under `@theme`:

Token	Value / role
`--color-maroon`	#5b1a2b — primary dark (footer, buttons, frames)
`--color-wine-deep`	#3d1019 — deepest maroon
`--color-gold / gold-hi / gold-deep`	#c39a4e / #e6cd8b / #a97e30 — accents & gradients
`--color-cream / cream-soft / surface`	page & card backgrounds (surface flips in dark mode)
`--color-ink / muted / light`	text tones (light = fixed ivory for text on maroon)
`--line / --line-strong / --frame`	border tones

Utilities: `.eyebrow` (uppercase gold label), `.gold-text` (gold gradient text). Dark mode via `:root[data-theme="dark"]`. Fonts: Cormorant Garamond (serif headings) + Jost (body).

13. Known gotchas & fixes

> **Tailwind v4 arbitrary values** sometimes don't generate in this dev setup (e.g. `w-[76px]`, `h-dvh`). When a size silently fails, use an **inline** `style` instead (the admin shell uses `height:100dvh` and fixed widths inline for this reason).

- **Windows Prisma EPERM** — `prisma generate`/migrate fails while the dev server holds the query engine. Stop the dev task, kill stray `next dev` processes, then migrate.

- **Dev-server restart** — stopping the task wrapper doesn't kill the underlying `next dev` PID (it keeps port 3000). Kill the PID before starting a fresh server.
- **Browser-pane transitions** — CSS transitions don't advance while the preview pane isn't painting, so size polling during an animation is unreliable; verify animated UI with a screenshot.
- **Server-action success + redirect** — actions can't fire client toasts; success paths redirect with `?flash=<key>` and `flash-toast.tsx` shows the toast on arrival, then strips the param.
- **SSE + in-memory bus** is process-local — perfect on a single Node/VPS host; multi-instance/serverless would need Redis pub/sub.

14. Current data state (development DB)

- Products: 15 · Categories: 4 · Customers: 3 (real) · Admin users: 1 (real)
- Orders: 17 total = 1 real + **16 demo** (order numbers prefixed `QATEST-`).

> **The 16 `QATEST-` orders are demo/seed data** added only to make the dashboard charts look populated (with line-items and statuses spread across the pipeline). They inflate Revenue / Orders / Top products / Category / Pipeline. Remove them (items cascade) with:
>
> ```
> node -e "const{PrismaClient}=require('@prisma/client');const p=new PrismaClient();p.order.deleteMany({where:{orderNumber:{startsWith:'QATEST-'}}}).then(r=>{console.log('removed',r.count);return p.$disconnect()})"
> ```

15. Deployment checklist (before go-live)

- Set a strong `AUTH_SECRET` and real `ADMIN_EMAIL` / `ADMIN_PASSWORD`; run `db:create-admin`.
- Add live **courier API keys** (Leopards / TCS) to enable real shipment booking.
- Add **payment gateway** keys (PayFast / Safepay) for real online payments — COD already works.
- Configure **SMTP** for order confirmation & admin-notify emails.
- Replace stock/editorial imagery with real **product photography**.
- Point `NEXT_PUBLIC_SITE_URL` at the production domain; run `npm run build`.
- Remove the `QATEST-` demo orders (see §14).
- Prefer a single-instance Node host / VPS so the real-time SSE notifications work without extra infra.

16. Pending / next tasks

- **Client-supplied for launch:** courier keys, payment keys, SMTP creds, product photos.
- **Ideas discussed:** "Today's snapshot" card, payment-method split, repeat-vs-new customers, conversion funnel (needs traffic analytics), a category donut.
- **Final QA:** full end-to-end test pass of the storefront + admin (the standing request after each feature set).